



武汉大学
WUHAN UNIVERSITY

m3u8流媒体格式数据获取实验

AI程序设计

2026.07



目 录

CONTENTS

01 基础知识

02 学习目标

03 具体步骤

04 拓展阅读



基础知识

01

□m3u8格式介绍

- m3u8是一种**视频流媒体格式**，通常用于在线播放视频，广泛用于 HLS (HTTP Live Streaming) 流媒体协议
- m3u8文件本身不是一段视频，而是一个**索引纯文本文件**，通常基于 M3U 的 UTF-8 编码扩展。打开它时会根据它的索引找到对应的音视频文件的网址 **(以ts格式保存)** 进行在线播放。
- **m3u8文件的典型结构如下：**

```
#EXTM3U 文件标识
#EXT-X-VERSION:3 协议版本
#EXT-X-ALLOW-CACHE:YES
#EXT-X-TARGETDURATION:20 最大分片时长
#EXT-X-MEDIA-SEQUENCE:0
#EXT-X-DISCONTINUITY
#EXTINF:20.000000, 分片时长
#EXT-X-PROGRAM-DATE-TIME:2022-11-29T16:01:53+0800 分片对应时间
recording 20221129154234/recording 20221129154234-57.ts 分片URL
#EXTINF:20.000000,
#EXT-X-PROGRAM-DATE-TIME:2022-11-29T16:02:13+0800
recording 20221129154234/recording 20221129154234-58.ts
#EXTINF:20.000000,
#EXT-X-PROGRAM-DATE-TIME:2022-11-29T16:02:33+0800
recording 20221129154234/recording 20221129154234-59.ts
#EXTINF:20.000000,
#EXT-X-PROGRAM-DATE-TIME:2022-11-29T16:02:53+0800
recording 20221129154234/recording 20221129154234-60.ts
```

通俗来说，网页完整视频被分割为若干个子视频分段，每个子片段以ts格式保存，m3u8文件负责记录所有子片段对应ts格式文件的名称以及对应时间段。



□ts文件URL拼接规则

将m3u8的基地址与每个ts片段自带参数进行拼接，例如：

1. 某m3u8的url为：

```
https://xigua-cdn.haima-zuida.com/20210219/19948_fcbbc225a/1000k/hls/index.m3u8
```

2. 它的基地址为：

```
https://xigua-cdn.haima-zuida.com/20210219/19948_fcbbc225a/1000k/hls
```

3. 与各个ts片段名称（如075a34ccdd000000.ts）拼接得到：

```
https://xigua-cdn.haima-zuida.com/20210219/19948_fcbbc225a/1000k/hls/075a34ccdd000000.ts
```

4. 基于ts文件对应url，下载所有的ts片段，将他们拼接整合即可得到完整视频



学习目标

02



□m3u8流媒体格式数据获取实验

具体步骤：

1. 分析视频网页，找到并下载其对应的m3u8文件
2. 解析m3u8文件，下载其索引的全部ts片段
3. 合并所有ts片段，并转换为mp4文件



1. **requests**: HTTP 请求库, 获取网页源代码或文件资源
2. **re (可选)**: 正则表达式, 用于匹配 URL 等关键信息 (也可使用urljoin自动化解析)
3. **BeautifulSoup (可选)**: HTML/XML 解析库, 适合提取页面中的结构化信息
4. **threading (可选)**: 多线程进程运行, 适合提升视频下载速度
5. **ffmpeg**: 视频合成工具, 将ts合并内容转化为mp4文件



BeautifulSoup用于解析 HTML 或 XML 数据, 得到页面中的结构化信息:

- `soup = BeautifulSoup(html, 'html.parser')` #初始化解析器

BeautifulSoup常见的操作包括查找标签、提取文本等:

- `soup.find()` #返回第一个匹配的标签
- `soup.find_all()` #返回所有匹配的标签
- `tag = soup.find('a')` # 查找第一个<a>标签
- `all_text = soup.get_text()` #获取页面中所有文本内容



正则化表达式是用于模式匹配的强力工具
通过特殊语法规则可以快速定位文本特征、提取结构化数据。

表达式	说明
.	除了\n和\r的所有字符
\d	数字
\D	非数字
\w	数字、字母和下划线
\W	非数字、字母和下划线
\s	空格（包括制表符、换页符等）
[a-z]	小写英文字母
[a-zA-Z0-9]	大小写英文字母与数字
[123]	数字123
[^123]	不是数字123



正则表达式

正则化表达式是用于模式匹配的强力工具
通过特殊语法规则可以快速定位文本特征、提取结构化数据。

表达式	说明
*	出现次数 ≥ 0
+	出现次数 ≥ 1
{n}	出现次数=n
{n,m}	$m \geq \text{出现次数} \geq n$
^	以开头
\$	以结尾
?	关闭贪婪模式
()	用于获取括号内匹配成功的字符串

正则化表达式是用于模式匹配的强力工具，通过特殊语法规则可以快速定位文本特征、提取结构化数据。

如定义密码的正则表达式：

规则：英文字母开头，可以包括数字、大小写英文字母、下划线，6-16位

```
password_pattern = r'^[a-zA-Z][a-zA-Z0-9_]{5,15}$'
```

匹配m3u8的正则表达式：

规则：http(s)开头，中间没有空格/引号等无效字符、后缀为.m3u8

```
m3u8_pattern = r'https?:/[^\s"']+?.m3u8'
```



```
import re
```

```
ret = re.search(【正则表达式】，【被提取的字符串】)
```

注：扫描整个字符串并返回第一个成功的匹配



```
import re
```

```
ret = re.findall(【正则表达式】，【被提取的字符串】)
```

注：找到所有满足匹配条件的结果，并以列表的形式返回



```
import re
```

```
ret = re.match(【正则表达式】，【被匹配的字符串】)
```

注：

如果匹配成功，返回<class 're.Match'>对象

如果匹配不成功，返回None



```
import re
```

```
ret = re.sub ( 【正则表达式】 , 【替换成的字符串】 , 【被匹配的字符串】 )
```



具体步骤

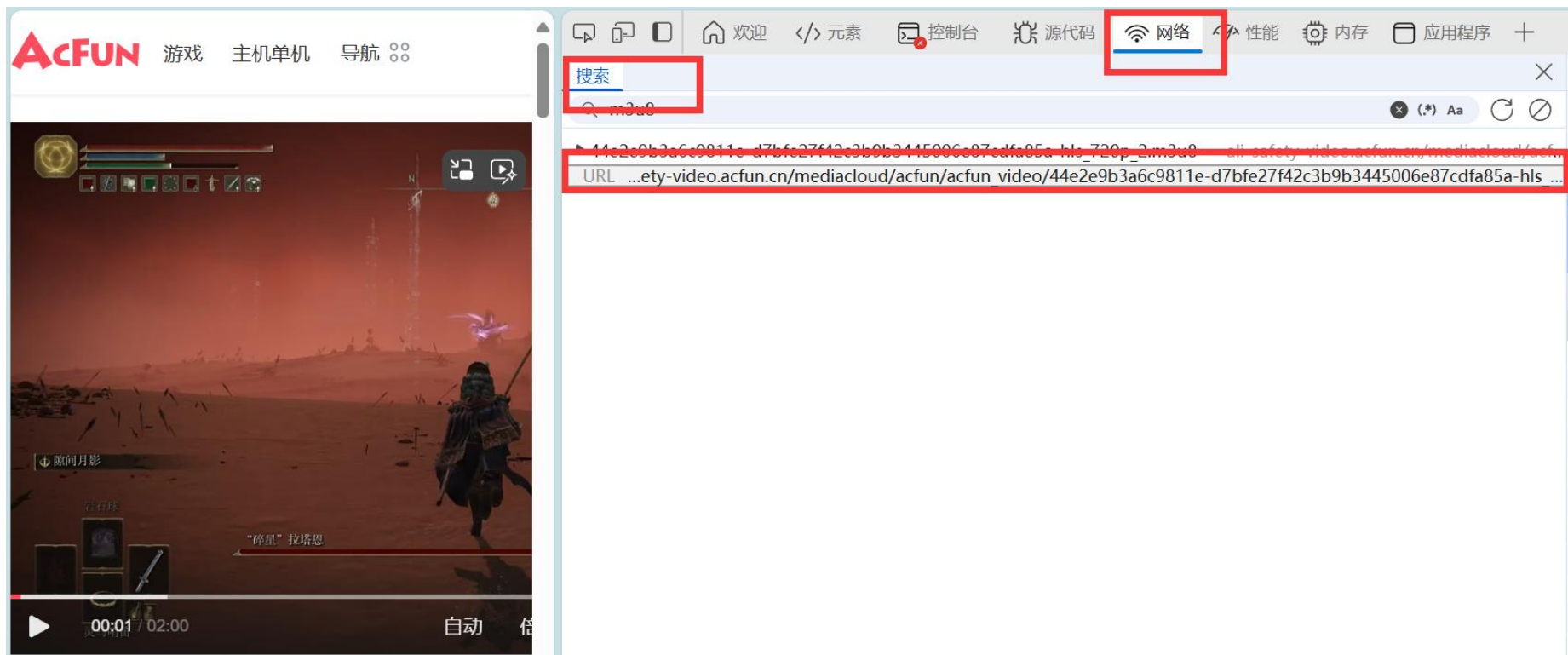
03



分析网站

以acfun任意视频网页为例，如： <https://www.acfun.cn/v/ac33856487>

F12->网络 (Network) ->搜索过滤"m3u8" -> 直接得到m3u8的url



此过程也可以基于Beautifulsoup提取html结构化信息，然后通过re正则化匹配得到



下载m3u8文件

伪装浏览器，访问m3u8 url，下载其中的内容

```
def download(url, name):  
    """ 下载ts文件 """  
    headers = {  
        'User-Agent': 'Mozilla/5.0 (Windows NT 6.3; Win64; x64; rv:84.0) Gecko/20100101 Firefox/84.0',  
        'Referer': 'https://www.acfun.cn/'  
    }  
    try:  
        r = requests.get(url, headers=headers, timeout=15)  
        if r.status_code == 200:  
            with open(name, "wb") as f:  
                f.write(r.content)  
            print(f"[下载成功] {name}")  
        else:  
            print(f"[失败] 状态码 {r.status_code}: {url}")  
    except Exception as e:  
        print(f"[异常] {url}: {str(e)}")
```

关键函数：requests.get(url, headers)
向服务器发送get请求并接收响应

headers用于模拟真实用户请求行为



解析m3u8文件

解析m3u8文件中的所有包含'.ts'的内容，基于urljoin自动拼接完整的ts子段对应url，保存在ts_list中。

```
from urllib.parse import urljoin
```

```
def parse_m3u8(m3u8_url, m3u8_local):  
    """ 解析M3U8文件, 返回TS列表 """  
    ts_list = []  
    with open(m3u8_local, "r") as f:  
        lines = f.readlines()  
  
    count = 0  
    for line_num, line in enumerate(lines, 1):  
        line = line.strip()  
        if line.startswith('#') or not line:  
            continue  
        # 允许TS路径包含查询参数 (如.ts?pkey=xxx)  
        if '.ts' in line.split('?')[0]: # 检查是否包含.ts  
            count += 1  
            full_url = urljoin(m3u8_url, line) # 自动拼接完整ts url  
            local_name = f"{count:05d}.ts" # 本地按序号命名, 如00001.ts  
            ts_list.append((count, line_num, line, full_url, local_name))  
    return ts_list
```



批量下载ts文件

多线程（可选）下载所有ts文件

```
def m3u8_download_multi_thread(m3u8, mp4, thread_max=10):  
    ts_dir = os.path.join(mp4, "ts")  
    os.makedirs(ts_dir, exist_ok=True)  
  
    # 下载M3U8文件  
    m3u8_local = os.path.join(mp4, "index.m3u8")  
    download(m3u8, m3u8_local)  
  
    # 解析TS列表（直接传递m3u8原始URL）  
    ts_list = parse_m3u8(m3u8, m3u8_local)  
  
    # 多线程下载  
    for idx, line_num, ts_rel, ts_url, local_name in ts_list:  
        while threading.active_count() >= thread_max:  
            time.sleep(1)  
        save_path = os.path.join(ts_dir, local_name)  
        print(f"队列 [{idx}/{len(ts_list)}] {ts_rel} -> {local_name}")  
        threading.Thread(target=download, args=(ts_url, save_path)).start()  
  
    # 等待所有线程完成  
    while threading.active_count() > 1:  
        time.sleep(1)
```




合并转换为mp4

将所有ts文件按时间顺序排序，然后合并，最后通过ffmpeg转化为mp4视频。

```
def ts_join(mp4):
    ts_dir = os.path.abspath(os.path.join(mp4, "ts")) # 绝对路径
    os.makedirs(ts_dir, exist_ok=True)
    output_mp4 = os.path.abspath(os.path.join(mp4, f"{os.path.basename(mp4)}.mp4"))
    # 生成文件列表（按下载顺序）
    ts_files = sorted(
        [os.path.join(ts_dir, f) for f in os.listdir(ts_dir) if f.endswith('.ts')],
        key=lambda x: int(os.path.splitext(os.path.basename(x))[0])
    )

    list_file = os.path.join(ts_dir, "filelist.txt")
    with open(list_file, 'w', encoding='utf-8') as f:
        for file in ts_files:
            f.write(f"file '{file}'\n") # 使用绝对路径

    # FFmpeg合并命令
    cmd = ['ffmpeg',
           '-f', 'concat',
           '-safe', '0',
           '-i', list_file,
           '-c', 'copy',
           '-y',
           output_mp4]

    try:
        subprocess.run(cmd, check=True)
        print(f"合并成功: {output_mp4}")
    except subprocess.CalledProcessError as e:
        print(f"合并失败: {e}")
```

注：不同系统（windows与linux）的合并命令有差别，这里为linux的concat指令



m3u8内容拓展:

[m3u8 文件格式详解 - 简书](#)

[某学院m3u8视频解密获取分析实战分享 - jpSpaceX - 博客园](#)



拓展练习

04



1. 自行实现任一网页m3u8流数据文件的视频爬取，要求调用beautifulsoup+re结构化匹配
2. 进一步爬取经过**加密**的m3u8流数据视频